



FULL STACK QUEST

3ª Edição — Do Java Zero ao Full Stack

EDIÇÃO COMPLETA · COMEÇA NO JAVA BÁSICO, TERMINA EM PRODUÇÃO

14

MUNDOS

44

FASES

14

BOSS FIGHTS

1

FINAL BOSS

Personagem: Desenvolvedor(a) Full Stack Java · Nível atual: Nível 1 (Aprendiz)



Como ler este mapa

Esta edição começa do zero absoluto em Java e vai até você colocar um sistema completo no ar. São **14 Mundos**, cada um fatiado em **Fases pequenas** para você nunca travar em um tema gigante demais. No fim de cada Mundo existe uma **Boss Fight**: um projeto prático, propositalmente simples, para fixar o que foi estudado. No final da jornada tem um **Final Boss** que junta tudo.

☐ Fase de estudo ☐ Loot Secreto (curiosidade / aprofundamento) ☐ Boss Fight (projeto prático)

💡 DICA DE PROGRESSÃO

Os primeiros 4 Mundos são só Java puro, sem Spring, sem banco, sem web — a ideia é sair sabendo a linguagem de verdade antes de misturar frameworks. Só no **Mundo 5** (A Ponte para o Full Stack) o Java começa a conversar com banco de dados e com a internet. Vá com calma: uma Fase por sessão de estudo já é um ótimo ritmo, e não tem problema nenhum revisar um Mundo antes de seguir para o próximo.

TRILHA COMPLETA

- | | |
|--------------------------------------|---|
| 1 Fundamentos de Java | 8 Spring Boot Profissional |
| 2 O Coração do Java (POO) | 9 Segurança & Testes |
| 3 Estruturas & Exceções | 10 Fundamentos de Frontend |
| 4 Java Moderno & Concorrência | 11 TypeScript & React |
| 5 A Ponte para o Full Stack | 12 Arquitetura & Design Patterns |
| 6 Banco de Dados Relacional | 13 Docker & CI/CD |
| 7 A Web por Dentro (HTTP) | 14 Cache, Mensageria & Cloud |



A Vila Inicial (Fundamentos de Java)

+60 XP

Todo herói começa numa vila pacata aprendendo os golpes básicos. Aqui você aprende o vocabulário puro da linguagem Java, sem pressa — é a base de tudo que vem depois.

1.1 Sintaxe básica

ESSENCIAL

O alfabeto da linguagem: sem isso, nada mais faz sentido.

Variáveis

Tipos primitivos e wrappers

Operadores lógicos

Operadores matemáticos

1.2 Estruturas de controle

ESSENCIAL

É o que dá "decisão" e "repetição" ao seu código.

if / else

switch

switch expressions

for

while

do-while

1.3 Strings e Arrays

PRÁTICA

Como guardar e manipular várias informações de uma vez, sem banco de dados ainda.

Manipulação de Strings

Arrays

Iterar e buscar valores

💡 LOOT SECRETO

switch expressions: o Java moderno permite escrever switch como uma expressão que retorna valor direto (com ->), sem precisar de break em cada linha. É mais curto e evita um bug clássico: esquecer o break e "vazar" para o próximo caso sem querer.

⚔️ BOSS FIGHT 1: A CALCULADORA DO APRENDIZ

Muito fácil

Missão: criar um programa de console com um menu de operações simples.

- Montar um menu (usando switch) com 4 operações matemáticas.
- Usar um while para o menu voltar a aparecer até o usuário escolher "sair".
- Guardar o histórico das últimas 5 contas em um Array e exibi-lo ao final.



O Coração do Java (POO)

+90 XP

Aqui mora a fundação de qualquer sistema robusto que você vai construir daqui pra frente — de um validador de CPF a um sistema completo de gestão.

2.1 Classes, objetos e encapsulamento

ESSENCIAL

A unidade básica de tudo em Java: como modelar "coisas do mundo real" em código.

Classes e Objetos

Construtores

this

private / public / protected

Getters e Setters

2.2 Herança e polimorfismo

ESSENCIAL

Como reaproveitar comportamento entre classes parecidas, sem repetir código.

Herança

Classes abstratas

@Override

super

Polimorfismo

2.3 Interfaces e Records

APROFUNDAMENTO

Interfaces são contratos; Records são a forma moderna de guardar dados imutáveis.

Interfaces

Estender x Implementar

Records

💡 LOOT SECRETO

Herdar nem sempre é a resposta: antes de usar herança, pergunte "isso É um tipo daquilo, ou apenas TEM um daquilo?". Um Carro não é um Motor, ele TEM um motor. Errar essa pergunta é a causa nº 1 de hierarquias de herança confusas mais pra frente.

🔪 BOSS FIGHT 2: O VALIDADOR DE CPF

Fácil

Missão: construir um pequeno sistema de cadastro de pessoas usando os 4 pilares da POO.

- Criar uma classe Pessoa com atributos privados e getters/setters.
- Criar Cliente e Funcionario herdando de Pessoa.
- Implementar uma interface Validavel com o método validarCpf().
- Criar um record simples PessoaResumo para exibir só nome e CPF formatado.



A Masmorra das Coleções

+90 XP

Sistemas de verdade lidam com muitos dados ao mesmo tempo e com coisas que dão errado. Aqui você aprende a guardar listas de informação na memória e a prever falhas antes que virem problema em produção.

3.1 Collections Framework

ESSENCIAL

Saber a diferença entre *List*, *Set* e *Map* é usado o tempo todo, em qualquer sistema.

List (ArrayList, LinkedList)

Set (HashSet)

Map (HashMap)

3.2 Tratamento de exceções

ESSENCIAL

Prever o que pode dar errado é o que separa um script de um sistema confiável.

try-catch-finally

Checked x Unchecked

Exceções personalizadas

3.3 Enums

BOAS PRÁTICAS

Muito melhor que usar texto solto para representar status ou categorias fixas.

Tipos enumerados

Enums com atributos e métodos

💡 LOOT SECRETO

HashMap não tem ordem garantida: se você precisa que a ordem de inserção seja mantida, use `LinkedHashMap`; se precisa que fique sempre ordenado por chave, use `TreeMap`. Usar `HashMap` comum e depender da ordem é uma fonte clássica de bugs "aleatórios".

🔪 BOSS FIGHT 3: O ESTOQUE QUE NÃO PERDOA

Fácil

Missão: montar um controle de estoque em memória (sem banco de dados ainda).

- Guardar os produtos em uma `List` e usar um `Map<String, Produto>` para busca rápida por código.
- Criar um Enum `StatusProduto` (`DISPONIVEL`, `VENCIDO`, `EM_FALTA`).
- Criar e lançar uma exceção personalizada `ProdutoVencidoException` ao tentar vender um item vencido.

📖 Se quiser, evolua essa ideia depois no seu `EstoqueSupermercado`.



O Grimório Moderno (Java Avançado)

+110 XP

Esses recursos diferenciam um código amador de um código profissional e otimizado. É a última parada de Java puro antes de partir para o mundo backend/web.

4.1 Lambdas e Streams API

ESSENCIAL

Filtrar, mapear e reduzir coleções escrevendo bem menos código.

Expressões Lambda

map / filter / reduce

Collectors

4.2 Optional & Date/Time API

BOAS PRÁTICAS

Duas ferramentas que evitam os bugs mais clássicos e chatos de se debugar.

Optional

NullPointerException

LocalDate / LocalDateTime

Fusos horários

4.3 Concorrência básica

INTRODUÇÃO

Só o essencial: entender a ideia de múltiplas tarefas ao mesmo tempo.

Threads

Requisições simultâneas

💡 LOOT SECRETO

Streams preguiçosas: operações como map e filter só executam de verdade quando você chama uma operação terminal, como collect ou forEach. Até lá, é só uma receita esperando para ser cozinhada.

🔪 BOSS FIGHT 4: A REFATORAÇÃO DO GRIMÓRIO

Fácil/Médio

Missão: pegar o Estoque do Mundo 3 e deixá-lo com uma "cara mais moderna".

- Trocar pelo menos 3 loops for por Streams equivalentes (ex: filtrar produtos vencidos).
- Trocar uma busca que pode não encontrar nada para retornar `Optional<Produto>`.
- Usar `LocalDate` para a data de validade e calcular quantos dias faltam para vencer.



A Ponte para o Full Stack

+130 XP

Este é o momento em que você abandona interfaces gráficas embutidas (como Swing) e começa a fazer o Java conversar com a internet e com bancos de dados de verdade.



Maven/Gradle



JDBC/JPA



Spring Boot



APIs REST

5.1 Gerenciadores de dependência

FUNDAMENTO

Como adicionar bibliotecas externas (como o próprio Spring) ao seu projeto.

Maven

Gradle

pom.xml

5.2 JDBC x JPA/Hibernate

ESSENCIAL

Entender a conexão raiz com o banco antes de usar a "mágica" do ORM.

JDBC puro

JPA/Hibernate

@Entity / @Table / @Id

5.3 Spring Boot essencial

ESSENCIAL

O principal framework do mercado Java para construir sistemas web.

Inversão de Controle (IoC)

Injeção de Dependência (DI)

5.4 APIs REST e DTO

ESSENCIAL

Como seu Java vai finalmente "falar" com um frontend.

@RestController

GET/POST/PUT/DELETE

Respostas em JSON

Padrão DTO

💡 LOOT SECRETO

A virada de chave: ao invés de o Java desenhar botões e janelas, ele passa a processar regras de negócio (calcular abastecimento, gerenciar estoque, administrar uma frota) e devolve só os dados puros — quem cuida do visual agora é a camada de frontend, em outro Mundo mais à frente.

🔪 BOSS FIGHT 5: O PRIMEIRO PORTAL

Médio

Missão: criar seu primeiro endpoint Java conectado a um banco de verdade.

- Criar um projeto Spring Boot via Maven com as dependências Web e JPA.
- Transformar sua classe Produto (do Mundo 3) em uma @Entity.
- Criar um @RestController com um endpoint GET que devolve a lista de produtos em JSON usando um DTO.



As Catacumbas de Dados

+120 XP

Todo bom sistema precisa de uma masmorra bem organizada para guardar seus tesouros. Aqui você domina SQL de verdade e troca bancos simples pelo PostgreSQL, usado no mercado.

6.1 SQL do zero ao avançado

ESSENCIAL

A linguagem universal para conversar com bancos relacionais.

SELECT/INSERT/UPDATE/DELETE

JOINS

GROUP BY / HAVING

Subqueries e CTE

6.2 Modelagem de dados

FUNDAMENTO

Um bom modelo evita 80% dos problemas antes de escrever a primeira linha de código.

Entidade e Relacionamento

Cardinalidade

Normalização (1FN, 2FN, 3FN)

Modelo ER

6.3 PostgreSQL na prática

MIGRAÇÃO

O banco relacional mais usado em projetos profissionais Java.

Instalação e clientes (DBBeaver/pgAdmin)

Índices

PRIMARY/FOREIGN KEY

Transactions

💡 LOOT SECRETO

Índice não é mágica grátis: ele acelera buscas (SELECT), mas deixa escritas (INSERT/UPDATE) um pouco mais lentas, porque o banco precisa atualizar o índice também. Por isso não se cria índice em tudo — só nas colunas realmente usadas em filtros e JOINS.

🗡️ BOSS FIGHT 6: A GRANDE MIGRAÇÃO

Médio

Missão: modelar e persistir de verdade o sistema do Mundo 5 em PostgreSQL.

- Desenhar um diagrama ER simples relacionando Produto, Fornecedor e Categoria.
- Criar as tabelas em PostgreSQL com chaves primárias e estrangeiras corretas.
- Escrever 3 consultas com JOIN para responder perguntas reais (ex: "produtos por fornecedor perto de vencer").



A Torre das Comunicações

+80 XP

Antes de deixar suas APIs mais robustas, vale entender de verdade a linguagem que os sistemas usam para se falar pela internet. Mundo curto, mas fundamental.

7.1 Como a web funciona

CONCEITUAL

Entender o caminho de uma requisição do navegador até o servidor.

HTTP / HTTPS

DNS

TCP/IP

Cookies e Sessions

CORS

7.2 Verbos e status codes

ESSENCIAL

É o vocabulário que você já usou no Mundo 5 e vai usar todo dia como dev backend.

GET / POST / PUT / PATCH / DELETE

2xx sucesso

4xx erro do cliente

5xx erro do servidor

7.3 JSON, REST e idempotência

ESSENCIAL

O formato e o estilo de arquitetura usados por praticamente toda API moderna.

JSON

REST / RESTful

Stateless

Idempotência

💡 LOOT SECRETO

A visão de raio-x do navegador: abra o DevTools (F12) e vá na aba "Network". Toda vez que um site carrega, você vê as requisições reais indo e voltando, com headers, status code e o JSON de resposta — a melhor forma de estudar HTTP na prática.

🔪 BOSS FIGHT 7: CAÇADA DE REQUISIÇÕES

Muito fácil

Missão: virar um investigador de requisições HTTP (sem escrever backend nesta fase).

- Abra o DevTools em 2 sites diferentes e identifique 5 requisições GET e 2 POST.
- Anote pelo menos 3 status codes diferentes que você encontrar e o que cada um significa.
- Teste sua própria API do Mundo 5 no Postman/Insomnia e observe o JSON de resposta.



A Forja do Spring Boot

+150 XP

Agora que a ponte foi atravessada, é hora de deixar sua API com cara de profissional: camadas bem separadas, tratamento de erros e documentação de verdade.

8.1 Camadas e contratos da API

ESSENCIAL

Separar bem as responsabilidades é o que diferencia um projeto de faculdade de um profissional.

Controller → Service → Repository

DTOs de entrada e saída

8.2 Validação e tratamento de erros

ESSENCIAL

Sua API precisa responder direito quando algo sai errado, não só quando dá certo.

Bean Validation (@Valid)

@ControllerAdvice

Exceções customizadas na Web

8.3 Relacionamentos e paginação

APROFUNDAMENTO

Sistemas reais têm relacionamentos entre entidades e listas grandes demais para devolver tudo de uma vez.

@OneToMany / @ManyToOne

Lazy x Eager

Paginação

8.4 Documentação de API

PROFISSIONALIZAÇÃO

Uma API sem documentação é muito mais difícil de outros times (ou você mesmo) usarem.

OpenAPI

Swagger UI

Versionamento de API

💡 LOOT SECRETO

DTO não é frescura: expor a Entity direto na API parece mais rápido, mas vaza detalhes internos do banco e trava a evolução do sistema. O DTO é a "carta de embaixada" — só o que o mundo externo precisa ver.

🔪 BOSS FIGHT 8: A BIBLIOTECA ARCANA

Médio

Missão: construir uma API REST completa de biblioteca (Livros + Usuários + Empréstimos).

- CRUD completo (GET, POST, PUT, DELETE) para Livros e Usuários.
- Relacionar Empréstimo com Livro e Usuário, com paginação na listagem de livros.
- Documentar tudo com Swagger e testar via Postman/Insomnia.



A Fortaleza Blindada

+150 XP

Para que sua API suporte um frontend em produção, ela precisa ser blindada contra acessos indevidos e testável sem depender de clicar na tela toda hora.

Spring
Security

JWT



JUnit 5



Mockito

9.1 Autenticação e Spring Security

ESSENCIAL

É como o "porteiro" do seu sistema decide quem entra e o que pode fazer.

Authentication x Authorization

Hash de senha (BCrypt)

Roles ("admin", "user")

9.2 Login com JWT

ESSENCIAL

O padrão mais comum para autenticar APIs stateless no mercado hoje.

Estrutura de um Token JWT

Filtros de segurança

Rotas protegidas por perfil

9.3 Testes automatizados

QUALIDADE

Garantir que a regra de negócio funciona sem reabrir o sistema a cada alteração.

JUnit (Unit Test)

Mockito (mocks)

MockMvc

💡 LOOT SECRETO

O paradoxo dos mocks: com Mockito, você cria "dublês" para o banco de dados. Isso significa testar a lógica do seu Service sem nunca tocar no PostgreSQL de verdade — como treinar em uma sala de simulação antes da batalha real.

🗡 BOSS FIGHT 9: O SISTEMA DE CHAMADOS PROTEGIDO

Médio

Missão: transformar a Biblioteca do Mundo 8 (ou uma nova API de Helpdesk) em algo seguro e testado.

- Criar rota de login que devolve um Token JWT.
- Proteger endpoints: só ADMIN pode deletar registros, USER só pode criar/consultar.
- Escrever testes de unidade para o Service de criação (com Mockito).



O Ateliê das Ilusões (Frontend I)

+120 XP

De nada adianta um backend poderoso se ninguém consegue interagir com ele. Hora de sair do terminal e dominar o que o usuário realmente vê e toca.

10.1 HTML semântico

FUNDAMENTO

Estrutura correta facilita acessibilidade e deixa o CSS muito mais fácil depois.

Tags semânticas

Forms e Inputs

Tables

10.2 CSS moderno

ESSENCIAL

A verdadeira "batalha" de todo frontend: alinhar as coisas do jeito certo.

Box Model

Flexbox

Grid

Responsive Design

10.3 JavaScript moderno

ESSENCIAL

A linguagem que dá vida à página — obrigatória para qualquer full stack.

let/const, arrow functions

Promises / async-await

DOM e Events

Fetch API

💡 LOOT SECRETO

Flexbox resolve 80% dos casos: antes de sair usando Grid para tudo, tente resolver o layout com Flexbox. O Grid entra quando você precisa de um layout bidimensional de verdade (linhas E colunas ao mesmo tempo).

🔪 BOSS FIGHT 10: A PÁGINA VIVA

Fácil

Missão: criar uma página HTML/CSS/JS puro que consome a API do Mundo 8.

- Montar um layout responsivo simples com Flexbox (cards de livro).
- Usar `fetch()` para buscar os livros na sua própria API Spring Boot.
- Exibir os dados na tela dinamicamente, sem recarregar a página.



O Domínio das Interfaces (React)

+150 XP

Com o fundamento pronto, é hora de construir interfaces modernas e conectá-las ao backend blindado no Mundo 9.



TypeScript



React



Routing



State/Context



11.1 TypeScript essencial

PONTE COM JAVA

Vindo de Java, essa parte vai parecer familiar — é tipagem forte no mundo JS.

Types e Interfaces

Generics

Enums



11.2 React fundamentos

ESSENCIAL

A base de tudo: como React pensa em componentes e reatividade.

Components e Props

useState

useEffect

Forms controlados



11.3 React conectado ao backend

INTEGRAÇÃO

É aqui que o frontend finalmente conversa com a API do Mundo 9.

Routing (React Router)

Context API

Rotas protegidas

💡 LOOT SECRETO

TypeScript é o "Java do front": ele adiciona tipagem forte ao JavaScript, com Interfaces e Generics parecidos com os que você já domina em Java. Evita bugs bizarros de "undefined não é uma função".



BOSS FIGHT 11: A INTERFACE DO CHAMADO

Médio

Missão: conectar um frontend em React + TypeScript ao backend protegido do Mundo 9.

- Tela de Login que envia dados para o Spring e guarda o Token JWT recebido.
- Dashboard organizado (Flexbox/Grid) listando os chamados vindos da API.
- Formulário que envia um POST para abrir um novo chamado.



O Santuário da Arquitetura

+100 XP

Aqui você começa a sair de desenvolvedor iniciante. Não é sobre aprender uma tecnologia nova, e sim sobre escrever código que qualquer outro dev consiga entender e manter.

12.1 SOLID na prática

FUNDAMENTO

5 princípios simples que evitam boa parte dos problemas de manutenção de código.

S - Single Responsibility

O - Open/Closed

L - Liskov

I - Interface Segregation

D - Dependency Inversion

12.2 Clean Code & Arquitetura em camadas

APROFUNDAMENTO

Nomes bons e responsabilidades bem separadas valem mais que qualquer framework.

Clean Code

Clean Architecture

MVC

12.3 Design Patterns essenciais

FERRAMENTAS

Só os padrões mais usados no dia a dia — não vale tentar decorar os 23 clássicos.

Repository Pattern

Factory

Strategy

Builder

💡 LOOT SECRETO

Cuidado com a "padritite": não caia na armadilha de tentar encaixar um Design Pattern em absolutamente tudo. Um padrão só faz sentido quando resolve um problema real de duplicação ou acoplamento.

✂ BOSS FIGHT 12: A GRANDE REFORMA

Médio

Missão: aplicar SOLID e um padrão de projeto em um sistema já existente seu.

- Escolher uma classe "faz-tudo" de um projeto seu e quebrá-la em responsabilidades menores.
- Aplicar o Repository Pattern separando o acesso a dados da regra de negócio.
- Documentar em um README o "antes e depois" da refatoração.

📁 Bom candidato: o GerenciadorDeEscola ou o FrotaGro.



A Frota de Contêineres

+130 XP

Sua aplicação está pronta, mas só roda na sua máquina. Hora de empacotar tudo para que rode igual em qualquer lugar do mundo.



Docker

Docker
Compose

CI/CD



GitHub Actions

13.1 Docker essencial

FUNDAMENTO

Isolar sua aplicação com tudo que ela precisa para rodar, em qualquer computador.

Image e Container

Dockerfile

Volume

13.2 Docker Compose

ESSENCIAL

Sobe backend, banco e frontend juntos com um único comando.

docker-compose.yml

Serviços interligados

13.3 Integração contínua

AUTOMAÇÃO

Automatizar build, teste e deploy a cada push, sem fazer nada manual.

GitHub Actions

Pipeline: build → test → deploy

💡 LOOT SECRETO

O contêiner imortal: o Docker isola sua aplicação com um mini sistema operacional próprio. Se rodou no container do seu PC, vai rodar *exatamente igual* na nuvem — acabou aquele clássico "na minha máquina funciona".

🔪 BOSS FIGHT 13: O SISTEMA EMPACOTADO

Médio

Missão: dockerizar o Sistema de Chamados construído nos Mundos 9 e 11.

- Dockerfile para o backend Spring Boot.
- docker-compose.yml subindo backend + PostgreSQL + frontend React juntos.
- Pipeline simples no GitHub Actions rodando os testes a cada push.



Os Reinos Distantes (Escala & Cloud)

+200 XP

O reino final. Aqui você conhece as ferramentas que fazem sistemas aguentarem muito uso ao mesmo tempo, e coloca tudo para rodar na nuvem.

14.1 Cache com Redis

PERFORMANCE

Evita bater no banco de dados toda vez, deixando o sistema muito mais rápido.

Key/Value

TTL

Cache invalidation

14.2 Mensageria (RabbitMQ)

APROFUNDAMENTO

O primeiro passo para aplicações distribuídas, processando coisas de forma assíncrona.

Producer / Consumer

Queue

Kafka (visão geral)

14.3 Cloud com AWS

DEPLOY REAL

Não precisa aprender a AWS inteira — só o suficiente para colocar seu sistema no ar.

EC2

S3

RDS

IAM

💡 LOOT SECRETO (BÔNUS: OBSERVABILIDADE)

Quando o sistema cresce, você passa a precisar de **Logs, Metrics e Tracing**. No ecossistema Java, o combo **Spring Boot Actuator + Prometheus + Grafana** é o caminho mais usado no mercado — vale conhecer depois de dominar o resto.

🐉 FINAL BOSS: O SISTEMA GLOBAL DE VENDAS

Desafiador

Missão: construir o projeto que resume toda a jornada, para mostrar em entrevistas.

- Spring Boot + PostgreSQL para Clientes, Produtos, Estoque e Pedidos.
- Redis fazendo cache das buscas de produtos.
- Frontend em React + TypeScript consumindo a API com autenticação JWT.
- TUDO orquestrado em Docker Compose, com pipeline de CI/CD configurada.

🏆 Equipe seu teclado mecânico e prepare as poções de energia (café).

Você começou lá no "Hello World" e chegou a um sistema completo em produção. A quest sempre continua — o resto agora é ganhar XP no mundo real.



Checklist de Progressão

Marque cada Boss Fight conforme for concluindo. Uma visão rápida de onde você está na jornada.

	MUNDO	BOSS FIGHT	XP
☐	1. Fundamentos de Java	A Calculadora do Aprendiz	+60
☐	2. Orientação a Objetos	O Validador de CPF	+90
☐	3. Estruturas & Exceções	O Estoque que Não Perdoa	+90
☐	4. Java Avançado	A Refatoração do Grimório	+110
☐	5. A Ponte para o Full Stack	O Primeiro Portal	+130
☐	6. Banco de Dados	A Grande Migração	+120
☐	7. HTTP & Web	Caçada de Requisições	+80
☐	8. Spring Boot Profissional	A Biblioteca Arcana	+150
☐	9. Segurança & Testes	O Sistema de Chamados Protegido	+150
☐	10. Frontend Fundamentos	A Página Viva	+120
☐	11. TypeScript & React	A Interface do Chamado	+150
☐	12. Arquitetura	A Grande Reforma	+100
☐	13. Docker & CI/CD	O Sistema Empacotado	+130
☐	14. Cloud & Escala	FINAL BOSS: Sistema Global de Vendas	+200

 LEMBRETE FINAL

Dominando até o fim do **Mundo 5** (A Ponte para o Full Stack), você já está pronto para começar a plugar tecnologias de frontend nas suas aplicações. Até o fim do **Mundo 9**, já tem uma base muito sólida para uma vaga de Java Backend Júnior. React, Cloud, mensageria e arquitetura avançada são o que te leva de Júnior para Pleno.